Uthao — React Frontend Setup Guide

A complete, minimal-but-not-plain React frontend for the Uthao ride-sharing backend, styled Uber-style: black pill buttons, white cards, colored status pills, one small Tailwind entry file instead of a hand-rolled stylesheet. Implemented and verified in `uthao-frontend/` — this guide documents exactly what's there, file for file.

Author's note: this guide assumes the backend from this repo — `api-gateway` on port `8080`, JWT auth with claims `userId` + `role` (`RIDER` / `DRIVER`), and the endpoints documented in `Docs/CONTRACTS.md`.

Table of Contents

---

1. Goals & Design Approach

- **Uber-style, not glary.** Black pill buttons, white cards on a light-gray backdrop, colored status pills, a system font stack. No gradients, no animation libraries, no component kit (MUI/Chakra/AntD) — utility classes instead.
- **One responsibility per file.** API calls live in `src/api/`, shared state lives in `src/context/`, pages live in `src/pages/`, dumb UI pieces live in `src/components/`.
- **Function components + Hooks only.** No class components, no Redux — Context API is enough for a project this size.
- **JSX everywhere.** All component files use the `.jsx` extension so Vite's React plugin transforms them automatically.

---

2. Tech Stack

Concern	Choice	Why
Build tool	**Vite**	Fast dev server, zero-config JSX/ESM, tiny prod bundles
UI library	**React 18** (function components + Hooks)	Matches the requirement, huge ecosystem
Routing	**react-router-dom v6**	Standard client-side routing, nested + protected routes
HTTP client	**axios**	Interceptors for attaching the JWT and handling 401s
Global state	**React Context API**	No extra dependency needed for auth/notifications state
Styling	**Tailwind CSS v4** (`@tailwindcss/vite`)	Utility classes instead of hand-rolled CSS — no separate `tailwind.config.js` needed in v4, just one `@import`
Map / location picking / live driver visibility	**Leaflet + react-leaflet v4** (OpenStreetMap tiles)	Riders don't know pickup/drop lat/lng — tap-to-pin on a free map instead of typing coordinates. The same map stack shows nearby available drivers once pickup is set. `react-leaflet@4` specifically, since v5 requires React 19 and this project is on React 18

No TypeScript in this guide (kept minimal per your request) — everything is plain `.jsx`/`.js`.

---

3. Prerequisites

Tool	Version
Node.js	18+ (20 LTS recommended)
npm	9+
The backend	`api-gateway` reachable at `http://localhost:8080` (see main `README.md`)

Confirm:

```
node -v npm -v
```

---

4. Project Structure (full tree)

The frontend lives as a sibling folder to the backend services:

```
Uthao/ ├── ...backend services (unchanged)... └── uthao-frontend/ ├── index.html ├── package.json ├── vite.config.js ├── .env ├── .env.example ├── .gitignore ├── README.md ├── src/ ├── main.jsx ├── App.jsx ├── index.css ├── api/ │ ├── client.js │ ├── authApi.js │ ├── riderApi.js │ ├── driverApi.js │ ├── matchingApi.js │ ├── tripApi.js │ ├── paymentApi.js │ ├── notificationApi.js ├── context/ │ ├── AuthContext.jsx │ ├── NotificationContext.jsx ├── hooks/ │ ├── useAuth.js │ ├── useNotifications.js ├── routes/ │ ├── AppRoutes.jsx │ ├── ProtectedRoute.jsx ├── components/ │ ├── Navbar.jsx │ ├── Button.jsx │ ├── Input.jsx │ ├── Card.jsx │ ├── Loader.jsx │ ├── StatusBadge.jsx │ ├── NotificationBell.jsx │ ├── LocationPickerModal.jsx │ ├── NearbyDriversMap.jsx ├── pages/ │ ├── LoginPage.jsx │ ├── RegisterPage.jsx │ ├── NotificationsPage.jsx │ ├── NotFoundPage.jsx ├── rider/ │ ├── RiderDashboard.jsx │ ├── NewRideRequest.jsx ├── driver/ │ ├── DriverOnboarding.jsx │ ├── DriverDashboard.jsx ├── trip/ │ ├── TripDetails.jsx ├── utils/ │ ├── constants.js
```

No `tailwind.config.js` or `postcss.config.js` — Tailwind v4's Vite plugin needs neither for a default theme. Every folder maps to exactly one concern: `api` talks to the backend, `context` holds shared state, `routes` decides what's visible, `components` are dumb/reusable, `pages` compose components + context + api into a screen.

---

5. Project Setup (commands)

From the `Uthao/` repo root:

```
npm create vite@latest uthao-frontend -- --template react cd uthao-frontend npm install npm install axios react-router-dom npm install tailwindcss @tailwindcss/vite npm install leaflet react-leaflet@4
```

Then wire Tailwind into Vite (`vite.config.js`) and replace the generated `index.css` with a single `@import "tailwindcss";` — both shown in §13.

---

6. Environment Variables

Vite exposes env vars prefixed with `VITE_` via `import.meta.env`.

`.env` (your local values — this repo commits it since it holds nothing secret):

```
VITE_API_BASE_URL=http://localhost:8080/api
```

`.env.example` (documents what's needed):

```
VITE_API_BASE_URL=http://localhost:8080/api
```

`.gitignore`

```
node_modules dist .env.local
```

All frontend traffic goes through the API Gateway (`/api/...`), exactly like Postman does in the backend README — never call a service's own port (8081–8087) directly from the browser.

---

7. Entry Point

`index.html`

```
<!doctype html> <html lang="en"> <head> <meta charset="UTF-8" /> <meta name="viewport" content="width=device-width, initial-scale=1.0" /> <title>Uthao</title> </head> <body> <div id="root"></div> <script type="module" src="/src/main.jsx"></script> </body> </html>
```

`src/main.jsx`

```
import React from "react"; import ReactDOM from "react-dom/client"; import App from "./App"; import "./index.css"; ReactDOM.createRoot(document.getElementById("root")).render( <React.StrictMode> <App /> </React.StrictMode> );
```

`src/App.jsx`

`App.jsx` is the composition root: it wraps the whole tree in the router, then in every context provider, then renders the layout (`Navbar` + routed page content, capped at a readable `max-w-2xl`).

```
import { BrowserRouter } from "react-router-dom"; import { AuthProvider } from "./context/AuthContext"; import { NotificationProvider } from "./context/NotificationContext"; import Navbar from "./components/Navbar"; import AppRoutes from "./routes/AppRoutes"; export default function App() { return ( <BrowserRouter> <AuthProvider> <NotificationProvider> <Navbar /> <main className="max-w-2xl mx-auto px-5 py-8 pb-16"> <AppRoutes /> </main> </NotificationProvider> </AuthProvider> </BrowserRouter> ); }
```

Provider order matters: `NotificationProvider` is nested inside `AuthProvider` because it will eventually need to know the logged-in user's id.

---

8. API Layer

One axios instance, configured once, reused everywhere. Every other API file only exports plain functions — no classes, no extra abstraction.

`src/api/client.js`

```
import axios from "axios"; const apiClient = axios.create({ baseURL: import.meta.env.VITE_API_BASE_URL, headers: { "Content-Type": "application/json" }, }); // Attach the JWT (if we have one) to every outgoing request. apiClient.interceptors.request.use((config) => { const token =
```

```
localStorage.getItem("uthao_token"); if (token) { config.headers.Authorization = `Bearer ${token}`; } return config; }); // Global 401 handling: token missing/invalid -> force back to login. apiClient.interceptors.response.use( (response) => response, (error) => { if (error.response?.status === 401) { localStorage.removeItem("uthao_token"); localStorage.removeItem("uthao_user"); localStorage.removeItem("uthao_driver_id"); window.location.href = "/login"; } return Promise.reject(error); } ); export default apiClient;
```

`src/api/authApi.js` — identity-service (`/api/auth/**`)

```
import apiClient from "./client"; export function registerUser({ name, email, password, role }) { return apiClient .post("/auth/register", { name, email, password, role }) .then((res) => res.data); } export function loginUser({ email, password }) { return apiClient.post("/auth/login", { email, password }).then((res) => res.data); } export function getCurrentUser() { return apiClient.get("/auth/me").then((res) => res.data); }
```

`src/api/riderApi.js` — rider-service (`/api/riders/**`)

```
import apiClient from "./client"; export function createRideRequest({ riderId, pickupLat, pickupLng, dropLat, dropLng }) { return apiClient .post("/riders/requests", { riderId, pickupLat, pickupLng, dropLat, dropLng }) .then((res) => res.data); } export function getRideRequests(riderId) { return apiClient.get(`/riders/${riderId}/requests`).then((res) => res.data); } export function cancelRideRequest(requestId) { return apiClient.post(`/riders/requests/${requestId}/cancel`).then((res) => res.data); }
```

`RideRequestResponse` only carries `id`, `riderId`, `status`, `createdAt` — no pickup/ drop coordinates come back from the API, so the rider's ride list doesn't try to display them.

`src/api/driverApi.js` — driver-service (`/api/drivers/**`)

```
import apiClient from "./client"; export function registerDriver({ userId, name, email, phone, licenseNumber }) { return apiClient .post("/drivers", { userId, name, email, phone, licenseNumber }) .then((res) => res.data); } export function addVehicle(driverId, { make, model, plateNumber, color }) { return apiClient .post(`/drivers/${driverId}/vehicles`, { make, model, plateNumber, color }) .then((res) => res.data); } export function updateDriverStatus(driverId, { status, currentLat, currentLng }) { return apiClient .patch(`/drivers/${driverId}/status`, { status, currentLat, currentLng }) .then((res) => res.data); } // Reads back the status set above. A driver who has never toggled status yet // has no DriverStatus row at all — the backend defaults that case to OFFLINE // rather than 404ing, since "no status set yet" is normal, not an error. export function getDriverStatus(driverId) { return apiClient.get(`/drivers/${driverId}/status`).then((res) => res.data); } export function getAvailableDrivers(lat, lng, radiusKm = 5.0) { return apiClient .get("/drivers/available", { params: { lat, lng, radiusKm } }) .then((res) => res.data); } export function getDriver(driverId) { return apiClient.get(`/drivers/${driverId}`).then((res) => res.data); } // Resolves driver-service's own Driver.id from the identity-service userId. // Every other driver-service endpoint is keyed by that id, not by userId. // 404 means this user hasn't completed driver onboarding yet. export function getDriverByUserId(userId) { return apiClient.get(`/drivers/by-user/${userId}`).then((res) => res.data); } export function getDriverTrips(driverId) { return apiClient.get(`/drivers/${driverId}/trips`).then((res) => res.data); }
```

`getDriverByUserId` and `getDriverStatus` both call endpoints that **did not exist in the original backend** — see §16. Every driver-service path (`status`, `vehicles`, `trips`) is keyed by driver-service's own `Driver.id`, never by the identity-service `userId`, so the frontend needs a way to go from one to the other on every login, not just right after registration. Separately, the original backend only let you *set* status (`PATCH .../status`) with no way to *read* it back

— meaning a page refresh had no way to know a driver's real current status.

`src/api/matchingApi.js` — matching-service (`/api/matching/**`)

```
import apiClient from "./client"; export function findDriver({ rideRequestId, riderId, pickupLat, pickupLng, dropLat, dropLng }) { return apiClient .post("/matching/find-driver", { rideRequestId, riderId, pickupLat, pickupLng, dropLat, dropLng, }) .then((res) => res.data); }
```

`src/api/tripApi.js` — trip-service (`/api/trips/**`)

```
import apiClient from "./client"; export function getTrip(tripId) { return apiClient.get(`/trips/${tripId}`).then((res) => res.data); } // Resolves the tripId trip-service creates asynchronously after a // driver.assigned event. 404s until the RabbitMQ listener has run — // callers should treat a 404 as "not created yet" and poll. export function getTripByRideRequest(rideRequestId) { return apiClient.get( `/trips/by-request/${rideRequestId}` ).then((res) => res.data); } export function startTrip(tripId) { return apiClient.post( `/trips/${tripId}/start` ).then((res) => res.data); } export function completeTrip(tripId) { return apiClient.post( `/trips/${tripId}/complete` ).then((res) => res.data); } export function cancelTrip(tripId) { return apiClient.post( `/trips/${tripId}/cancel` ).then((res) => res.data); }
```

`src/api/paymentApi.js` — payment-service (`/api/payments/**`)

```
import apiClient from "./client"; export function getPaymentByTrip(tripId) { return apiClient.get(`/payments/trip/${tripId}`).then((res) => res.data); } export function refundPayment(paymentId, reason) { return apiClient.post(`/payments/${paymentId}/refund`, { reason }).then((res) => res.data); }
```

`POST /payments/{paymentId}/refund` requires a `reason` in the body (`RefundRequest`) — note this takes `payment.id`, not `trip.id`.

`src/api/notificationApi.js` — notification-service (`/api/notifications/**`)

```
import apiClient from "./client"; export function getNotifications(userId) { return apiClient.get( `/notifications/user/${userId}` ).then((res) => res.data); } export function markNotificationRead(id) { return apiClient.patch( `/notifications/${id}/read` ).then((res) => res.data); }
```

---

9. Context API

`src/context/AuthContext.jsx`

Holds the logged-in user, the JWT, and — for drivers — driver-service's own `driverId`, all persisted to `localStorage` so a page refresh doesn't lose them. `login()` resolves `driverId` via `getDriverByUserId`; `register()` never does (a brand-new account can't have one yet); `completeDriverOnboarding()` sets it once onboarding finishes.

```
import { createContext, useState, useCallback } from "react"; import { loginUser, registerUser } from "../api/authApi"; import { getDriverByUserId } from "../api/driverApi";
```

```
export const AuthContext = createContext(null); const TOKEN_KEY = "uthao_token"; const USER_KEY = "uthao_user"; const DRIVER_ID_KEY = "uthao_driver_id"; export function AuthProvider({ children }) { const [user, setUser] = useState(() => { const stored = localStorage.getItem(USER_KEY); return stored ? JSON.parse(stored) : null; }); // driver-service's own Driver.id — every driver-service endpoint is keyed by // this, not by the identity-service userId. Null until resolved/onboarded. const [driverId, setDriverId] = useState(() => { const stored = localStorage.getItem(DRIVER_ID_KEY); return stored ? Number(stored) : null; }); const [loading, setLoading] = useState(false); const persistAuth = useCallback((authResponse) => { const { token, userId, name, role } = authResponse; localStorage.setItem(TOKEN_KEY, token); const nextUser = { userId, name, role }; localStorage.setItem(USER_KEY, JSON.stringify(nextUser)); // Reset any driverId left over from a previous account in this browser. localStorage.removeItem(DRIVER_ID_KEY); setUser(nextUser); setDriverId(null); return nextUser; }, []); const resolveDriverId = useCallback(async (userId) => { try { const driver = await getDriverByUserId(userId); localStorage.setItem(DRIVER_ID_KEY, String(driver.id)); setDriverId(driver.id); return driver.id; } catch { return null; } }, []); const login = useCallback( async (email, password) => { setLoading(true); try { const data = await loginUser({ email, password }); const nextUser = persistAuth(data); const resolvedDriverId = nextUser.role === "DRIVER" ? await resolveDriverId(nextUser.userId) : null; return { ...nextUser, driverId: resolvedDriverId }; } finally { setLoading(false); } }, [persistAuth, resolveDriverId] ); const register = useCallback( async (name, email, password, role) => { setLoading(true); try { const data = await registerUser({ name, email, password, role }); const nextUser = persistAuth(data); return { ...nextUser, driverId: null }; } finally { setLoading(false); } }, [persistAuth] ); // Called once driver onboarding (POST /drivers + vehicle) succeeds. const completeDriverOnboarding = useCallback((newDriverId) => { localStorage.setItem(DRIVER_ID_KEY, String(newDriverId)); setDriverId(newDriverId); }, []); const logout = useCallback(() => { localStorage.removeItem(TOKEN_KEY); localStorage.removeItem(USER_KEY); localStorage.removeItem(DRIVER_ID_KEY); setUser(null); setDriverId(null); }, []); const value = { user, driverId, loading, login, register, logout, completeDriverOnboarding }; return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>; }
```

src/hooks/useAuth.js

```
import { useContext } from "react"; import { AuthContext } from "../context/AuthContext"; export function useAuth() { const ctx = useContext(AuthContext); if (!ctx) { throw new Error("useAuth must be used inside <AuthProvider>"); } return ctx; }
```

src/context/NotificationContext.jsx

```
import { createContext, useState, useCallback } from "react"; import { getNotifications, markNotificationRead } from "../api/notificationApi"; export const NotificationContext = createContext(null); export function NotificationProvider({ children }) { const [notifications, setNotifications] = useState([]); const [loading, setLoading] = useState(false); const fetchNotifications = useCallback(async (userId) => { setLoading(true); try { const data = await getNotifications(userId); setNotifications(data); } finally { setLoading(false); } }, []); const markRead = useCallback(async (id) => { await markNotificationRead(id); setNotifications((prev) => prev.map((n) => (n.id === id ? { ...n, isRead: true } : n))); }, []); const unreadCount = notifications.filter((n) => !n.isRead).length; const value = { notifications, unreadCount, loading, fetchNotifications, markRead }; return ( <NotificationContext.Provider value={value}>{children}</NotificationContext.Provider> ); }
```

src/hooks/useNotifications.js

```
import { useContext } from "react"; import { NotificationContext } from "../context/NotificationContext"; export function useNotifications() { const ctx = useContext(NotificationContext); if (!ctx) { throw new Error("useNotifications must be used
```

```
inside <NotificationProvider>"); } return ctx; }
```

---

10. Routing

`src/routes/ProtectedRoute.jsx`

Guards a group of routes: no user → redirect to `/login`; wrong role → redirect home. Uses React Router's `<Outlet />` so it can wrap multiple child routes at once.

```
import { Navigate, Outlet } from "react-router-dom"; import { useAuth } from "../hooks/useAuth"; export default function ProtectedRoute({ allowedRoles }) { const { user } = useAuth(); if (!user) { return <Navigate to="/login" replace />; } if (allowedRoles && !allowedRoles.includes(user.role)) { return <Navigate to="/" replace />; } return <Outlet />; }
```

`src/routes/AppRoutes.jsx`

The root `/` redirect now checks `driverId` too — a DRIVER with no `driverId` yet gets sent to onboarding, not the dashboard.

```
import { Routes, Route, Navigate } from "react-router-dom"; import ProtectedRoute from "./ProtectedRoute"; import LoginPage from "../pages/LoginPage"; import RegisterPage from "../pages/RegisterPage"; import NotificationsPage from "../pages/NotificationsPage"; import NotFoundPage from "../pages/NotFoundPage"; import RiderDashboard from "../pages/rider/RiderDashboard"; import NewRideRequest from "../pages/rider/NewRideRequest"; import DriverOnboarding from "../pages/driver/DriverOnboarding"; import DriverDashboard from "../pages/driver/DriverDashboard"; import TripDetails from "../pages/trip/TripDetails"; import { useAuth } from "../hooks/useAuth"; export default function AppRoutes() { const { user, driverId } = useAuth(); return ( <Routes> <Route path="/login" element={<LoginPage />} /> <Route path="/register" element={<RegisterPage />} /> <Route element={<ProtectedRoute allowedRoles={["RIDER"]} />}> <Route path="/rider" element={<RiderDashboard />} /> <Route path="/rider/new" element={<NewRideRequest />} /> </Route> <Route element={<ProtectedRoute allowedRoles={["DRIVER"]} />}> <Route path="/driver/onboarding" element={<DriverOnboarding />} /> <Route path="/driver" element={<DriverDashboard />} /> </Route> <Route element={<ProtectedRoute />}> <Route path="/trips/:tripId" element={<TripDetails />} /> <Route path="/notifications" element={<NotificationsPage />} /> </Route> <Route path="/" element={ !user ? ( <Navigate to="/login" replace /> ) : user.role === "DRIVER" ? ( <Navigate to={driverId ? "/driver" : "/driver/onboarding"} replace /> ) : ( <Navigate to="/rider" replace /> ) } /> <Route path="*" element={<NotFoundPage />} /> </Routes> ); }
```

Route	Access	Page
`/login`	public	`LoginPage`
`/register`	public	`RegisterPage`
`/rider`	RIDER only	`RiderDashboard`
`/rider/new`	RIDER only	`NewRideRequest`

`/driver/onboarding`	DRIVER only	`DriverOnboarding`
`/driver`	DRIVER only	`DriverDashboard`
`/trips/:tripId`	any logged-in user	`TripDetails`
`/notifications`	any logged-in user	`NotificationsPage`
`/`	redirects by role (+ driverId)	—
`*`	public	`NotFoundPage`

---

11. Reusable Components

Every dynamic class string (button variant, status color) is a lookup in a plain JS object of **full, literal** Tailwind class names — Tailwind's build-time scanner only picks up classes it can see as complete strings in source, so `` `bg-${color}-50` `` silently wouldn't work. A `{ primary: "...", secondary: "..." }` map sidesteps that.

`src/components/Button.jsx`

```
const VARIANTS = { primary: "px-6 py-3.5 bg-black text-white hover:bg-neutral-800", secondary: "px-6 py-3.5 bg-white text-black border border-black hover:bg-neutral-100", ghost: "px-3 py-2 bg-transparent text-black hover:bg-neutral-100", }; export default function Button({ children, variant = "primary", full = false, className = "", ...rest }) { return ( <button className={`inline-flex items-center justify-center rounded-full text-sm font-bold transition-colors disabled:opacity-50 disabled:cursor-not-allowed ${VARIANTS[variant]} ${full ? "w-full" : ""} ${className}`} {...rest} > {children} </button> ); }
```

`src/components/Input.jsx`

```
export default function Input({ label, id, className = "", ...rest }) { return ( <div className="flex flex-col gap-1.5 mb-4"> {label && ( <label htmlFor={id} className="text-xs font-semibold text-neutral-600"> {label} </label> )} <input id={id} className={`px-3.5 py-3 border-[1.5px] border-neutral-200 rounded-md text-[15px] bg-white text-black focus:outline-none focus:border-black transition-colors ${className}`} {...rest} /> </div> ); }
```

`src/components/Card.jsx`

```
export default function Card({ title, children }) { return ( <div className="bg-white border border-neutral-200 rounded-2xl p-6 shadow-sm"> {title && <h3 className="text-base font-bold mb-4">{title}</h3>} <div>{children}</div> </div> ); }
```

`src/components/Loader.jsx`

Tailwind ships `animate-spin` by default, so the spinner needs no custom `@keyframes`.

```
export default function Loader() { return ( <div className="flex justify-center py-8" role="status" aria-label="Loading"> <span className="w-7 h-7 rounded-full border-[3px] border-neutral-200 border-t-black animate-spin" /> </div> ); }
```

`src/components/StatusBadge.jsx`

Every status string returned anywhere in the backend (`RideRequest`, `MatchResultDto`, `Trip`, `Payment`, `DriverStatus`) is mapped explicitly — green for the "good" states, amber for "in progress", red for "stopped", gray as the fallback.

```
const BADGE_STYLES = { AVAILABLE: "bg-green-50 text-green-700", MATCHED: "bg-green-50 text-green-700", COMPLETED: "bg-green-50 text-green-700", SUCCESS: "bg-green-50 text-green-700", ONGOING: "bg-amber-50 text-amber-700", REQUESTED: "bg-amber-50 text-amber-700", PENDING: "bg-amber-50 text-amber-700", CANCELLED: "bg-red-50 text-red-700", OFFLINE: "bg-red-50 text-red-700", NO_DRIVER_FOUND: "bg-red-50 text-red-700", REFUNDED: "bg-neutral-100 text-neutral-600", }; export default function StatusBadge({ status }) { const colors = BADGE_STYLES[status] ?? "bg-neutral-100 text-neutral-600"; return ( <span className={`inline-block px-2.5 py-1 rounded-full text-xs font-bold capitalize ${colors}`} > {String(status).replace(/_/g, " ")} </span> ); }
```

`src/components/NotificationBell.jsx`

```
import { Link } from "react-router-dom"; import { useNotifications } from "../hooks/useNotifications"; export default function NotificationBell() { const { unreadCount } = useNotifications(); return ( <Link to="/notifications" className="relative inline-flex text-lg" aria-label="Notifications" > <span>🔔</span> {unreadCount > 0 && ( <span className="absolute -top-1.5 -right-2 bg-red-600 text-white text-[10px] font-bold px-1.5 py-0.5 rounded-full min-w-[16px] text-center leading-none"> {unreadCount} </span> )} </Link> ); }
```

`src/components/LocationPickerModal.jsx`

Riders don't know pickup/drop latitude/longitude, so this replaces raw numeric inputs with a tap-to-drop-a-pin map (Leaflet + free OpenStreetMap tiles, no API key). Used by `NewRideRequest.jsx` for both pickup and drop-off, opened as a full-screen modal.

Three Leaflet-specific gotchas handled here:

- **Default marker icon path breaks under any bundler** — Leaflet's built-in icon URLs are relative to its own package location, which Vite doesn't preserve. Fixed by importing the three icon PNGs directly (Vite turns them into proper asset URLs) and reassigning `L.Icon.Default`'s options.
- **Click handling** goes through `useMapEvents` in a child component rendered inside `<MapContainer>` — react-leaflet doesn't expose map events as a prop on the container itself.
- **`<MapContainer center>` only applies once, on mount.** Geolocation resolves asynchronously, so setting `center` state *after* the map has already mounted on a fallback point wouldn't move it — react-leaflet doesn't watch that prop for changes. Instead the component withholds rendering `<MapContainer>` at all until `center` is known (geolocation success, geolocation failure → Dhaka fallback, or an already-known `initial` point), showing a brief "Finding your location…" state in the meantime so the map always mounts already centered correctly.

A blue `CircleMarker` marks the rider's actual detected position ("you are here"), kept visually distinct from the black `Marker` pin used for the point they're selecting — they're not always the same point (e.g. picking a drop-off far from where you're standing).

```
import { useState, useEffect } from "react"; import { MapContainer, TileLayer, Marker, CircleMarker, useMapEvents } from "react-leaflet"; import L from "leaflet"; import "leaflet/dist/leaflet.css"; import markerIcon2x from "leaflet/dist/images/marker-icon-2x.png"; import markerIcon from "leaflet/dist/images/marker-icon.png"; import markerShadow from "leaflet/dist/images/marker-shadow.png"; import Button from "./Button"; // Vite serves these as built asset URLs; Leaflet's default icon paths break // under any bundler unless explicitly reassigned like this. delete L.Icon.Default.prototype._getIconUrl; L.Icon.Default.mergeOptions({ iconRetinaUrl: markerIcon2x, iconUrl: markerIcon, shadowUrl: markerShadow, }); const DHAKA_CENTER = [23.8103, 90.4125]; const GEOLOCATION_OPTIONS = { timeout: 5000, maximumAge: 60000 }; function ClickCapture({ onPick }) { useMapEvents({ click(e) { onPick({ lat: e.latlng.lat, lng: e.latlng.lng }); }, }); return null; } export default function LocationPickerModal({ title, initial, onConfirm, onClose }) { const [point, setPoint] = useState(initial ?? null); // Map's initial center — react-leaflet only reads this once, on mount, so we // wait for geolocation to resolve before rendering <MapContainer> at all // rather than mounting on Dhaka and trying to recenter afterward. const [center, setCenter] = useState(initial ? [initial.lat, initial.lng] : null); const [userLocation, setUserLocation] = useState(null); useEffect(() => { if (initial) return; navigator.geolocation.getCurrentPosition( (position) => { const here = [position.coords.latitude, position.coords.longitude]; setUserLocation(here); setCenter(here); }, () => setCenter(DHAKA_CENTER), GEOLOCATION_OPTIONS ); }, [initial]); return ( <div className="fixed inset-0 bg-black/50 flex items-center justify-center z-50 p-4"> <div className="bg-white rounded-2xl w-full max-w-lg overflow-hidden shadow-lg"> <div className="px-5 py-4 border-b border-neutral-200 flex items-center justify-between"> <h3 className="text-base font-bold">{title}</h3> <button type="button" onClick={onClose} className="text-neutral-400 hover:text-black text-xl leading-none" aria-label="Close" > × </button> </div> <div className="h-80 w-full flex items-center justify-center bg-neutral-50"> {!center ? ( <p className="text-sm text-neutral-400">Finding your location…</p> ) : ( <MapContainer center={center} zoom={14} style={{ height: "100%", width: "100%" }}> <TileLayer attribution='&copy; <a href="https://www.openstreetmap.org/copyright">OpenStreetMap</a> contributors' url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png" /> <ClickCapture onPick={setPoint} /> {userLocation && ( <CircleMarker center={userLocation} radius={7} pathOptions={{ color: "#2563eb", fillColor: "#3b82f6", fillOpacity: 0.9, weight: 2 }} /> )} {point && <Marker position={[point.lat, point.lng]} />} </MapContainer> )} </div> <div className="px-5 py-4 flex items-center justify-between gap-3"> <p className="text-sm text-neutral-600"> {point ? `${point.lat.toFixed(5)}, ${point.lng.toFixed(5)}` : "Tap the map to drop a pin"} </p> <div className="flex gap-2 shrink-0"> <Button variant="secondary" onClick={onClose}> Cancel </Button> <Button disabled={!point} onClick={() => onConfirm(point)}> Use this location </Button> </div> </div> </div> ); }
```

`src/components/NearbyDriversMap.jsx`

Answers "when a driver is available, can the rider actually see them?" — before this, `getAvailableDrivers` existed in the API layer (§8) but no page ever called it. Renders once `pickup` is set on `NewRideRequest.jsx`: a read-only map centered on the pickup pin, with a car-emoji marker per driver returned by `GET /drivers/available`, refreshed every 10s so it reflects the periodic location pushes `DriverDashboard.jsx` now sends while a driver is `AVAILABLE` (see below).

```
import { useEffect, useState } from "react"; import { MapContainer, TileLayer, Marker, Popup } from "react-leaflet"; import L from "leaflet"; import "leaflet/dist/leaflet.css"; import markerIcon2x from "leaflet/dist/images/marker-icon-2x.png"; import markerIcon from "leaflet/dist/images/marker-icon.png"; import markerShadow from "leaflet/dist/images/marker-shadow.png"; import { getAvailableDrivers } from "../api/driverApi"; delete L.Icon.Default.prototype._getIconUrl; L.Icon.Default.mergeOptions({ iconRetinaUrl: markerIcon2x, iconUrl: markerIcon, shadowUrl: markerShadow, }); const carIcon = new L.DivIcon({ className: "", html: '<div style="background:#000;color:#fff;border-radius:9999px;width:28px;height:28px;display:flex;align-items:center;justify-content:center;font-size:16px;box-shadow:0 2px 4px rgba(0,0,0,.35)">🚗</div>', iconSize: [28, 28], iconAnchor: [14, 14], }); const RADIUS_KM = 5; const REFRESH_INTERVAL_MS = 10000; export default function NearbyDriversMap({ pickup }) { const [drivers, setDrivers] = useState([]); const [loading, setLoading] =
```

```
useState(true); useEffect(() => { if (!pickup) return; let cancelled = false; function load() { getAvailableDrivers(pickup.lat, pickup.lng, RADIUS_KM).then((data) => { if (!cancelled) { setDrivers(data); setLoading(false); } }); } setLoading(true); load(); const interval = setInterval(load, REFRESH_INTERVAL_MS); return () => { cancelled = true; clearInterval(interval); }; }, [pickup?.lat, pickup?.lng]); if (!pickup) { return null; } return ( <div className="mb-4"> <div className="flex items-center justify-between mb-1.5"> <span className="text-xs font-semibold text-neutral-600">Nearby drivers</span> <span className="text-xs text-neutral-400"> {loading ? "Searching…" : `${drivers.length} within ${RADIUS_KM}km`} </span> </div> <div className="h-48 w-full rounded-md overflow-hidden border-[1.5px] border-neutral-200"> <MapContainer center={[pickup.lat, pickup.lng]} zoom={13} scrollWheelZoom={false} style={{ height: "100%", width: "100%" }} > <TileLayer attribution='&copy; <a href="https://www.openstreetmap.org/copyright">OpenStreetMap</a> contributors' url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png" /> <Marker position={[pickup.lat, pickup.lng]} /> {drivers.map((d) => ( <Marker key={d.driverId} position={[d.currentLat, d.currentLng]} icon={carIcon}> <Popup> {d.name} · {d.vehiclePlate} <br /> {d.distanceKm.toFixed(1)} km away </Popup> </Marker> ))} </MapContainer> </div> </div> ); }
```

`src/components/Navbar.jsx`

The ghost-button class string is shared between a `<button>` (log out) and a `<Link>` (log in) — both need identical styling, so it's pulled into one constant rather than duplicated inline.

```
import { Link, useNavigate } from "react-router-dom"; import { useAuth } from "../hooks/useAuth"; import NotificationBell from "./NotificationBell"; const GHOST_BTN = "inline-flex items-center justify-center rounded-full text-sm font-bold px-3 py-2 bg-transparent text-black hover:bg-neutral-100 transition-colors no-underline"; export default function Navbar() { const { user, logout } = useAuth(); const navigate = useNavigate(); function handleLogout() { logout(); navigate("/login"); } return ( <header className="flex items-center justify-between px-8 py-3.5 bg-white border-b border-neutral-200 sticky top-0 z-10"> <Link to="/" className="text-xl font-extrabold tracking-tight text-black no-underline"> Uthao </Link> <nav className="flex items-center gap-5"> {user && <NotificationBell />} {user && ( <span className="flex items-center gap-2 text-sm text-neutral-600"> <span className="inline-flex items-center justify-center w-7 h-7 rounded-full bg-black text-white text-xs font-bold shrink-0"> {user.name.charAt(0).toUpperCase()} </span> {user.name} </span> )} {user ? ( <button className={GHOST_BTN} onClick={handleLogout}> Log out </button> ) : ( <Link to="/login" className={GHOST_BTN}> Log in </Link> )} </nav> </header> ); }
```

---

12. Pages

`src/utils/constants.js`

```
export const ROLES = { RIDER: "RIDER", DRIVER: "DRIVER" }; export const DRIVER_STATUS = { AVAILABLE: "AVAILABLE", OFFLINE: "OFFLINE" };
```

`src/pages/LoginPage.jsx`

Routes DRIVER logins to `/driver` if `driverId` resolved, `/driver/onboarding` otherwise — `login()` returns `driverId` directly so there's no stale-state read.

```
import { useState } from "react"; import { useNavigate, Link } from "react-router-dom"; import { useAuth } from "../hooks/useAuth"; import Input from "../components/Input"; import Button from "../components/Button"; export default function LoginPage() { const { login, loading } = useAuth(); const navigate = useNavigate(); const [form, setForm] = useState({
```

```
email: "", password: "" }); const [error, setError] = useState(""); function handleChange(e) { setForm({ ...form, [e.target.name]: e.target.value }); } async function handleSubmit(e) { e.preventDefault(); setError(""); try { const result = await login(form.email, form.password); if (result.role === "DRIVER") { navigate(result.driverId ? "/driver" : "/driver/onboarding"); } else { navigate("/rider"); } } catch { setError("Invalid email or password."); } } return ( <div className="min-h-[calc(100vh-65px)] flex items-center justify-center px-4 py-8"> <div className="w-full max-w-sm bg-white border border-neutral-200 rounded-2xl p-8 shadow-sm"> <h1 className="text-2xl font-extrabold mb-1.5">Welcome back</h1> <p className="text-neutral-600 text-sm mb-6">Log in to request or drive a ride.</p> <form onSubmit={handleSubmit}> <Input id="email" name="email" label="Email" type="email" value={form.email} onChange={handleChange} required /> <Input id="password" name="password" label="Password" type="password" value={form.password} onChange={handleChange} required /> {error && ( <p className="bg-red-50 text-red-600 px-3 py-2.5 rounded-md text-sm -mt-1 mb-4"> {error} </p> )} <Button type="submit" disabled={loading} full> {loading ? "Logging in…" : "Log in"} </Button> </form> <p className="text-center text-sm text-neutral-600 mt-4"> No account?{" "} <Link to="/register" className="text-black font-bold no-underline"> Sign up </Link> </p> </div> </div> ); }
```

`src/pages/RegisterPage.jsx`

The role picker is a two-way segmented control instead of a `<select>` — closer to how Uber's own signup reads, and avoids a dropdown for a binary choice.

```
import { useState } from "react"; import { useNavigate, Link } from "react-router-dom"; import { useAuth } from "../hooks/useAuth"; import Input from "../components/Input"; import Button from "../components/Button"; import { ROLES } from "../utils/constants"; export default function RegisterPage() { const { register, loading } = useAuth(); const navigate = useNavigate(); const [form, setForm] = useState({ name: "", email: "", password: "", role: ROLES.RIDER }); const [error, setError] = useState(""); function handleChange(e) { setForm({ ...form, [e.target.name]: e.target.value }); } function selectRole(role) { setForm({ ...form, role }); } async function handleSubmit(e) { e.preventDefault(); setError(""); try { const result = await register(form.name, form.email, form.password, form.role); navigate(result.role === "DRIVER" ? "/driver/onboarding" : "/rider"); } catch { setError("Could not register. Check your details and try again."); } } return ( <div className="min-h-[calc(100vh-65px)] flex items-center justify-center px-4 py-8"> <div className="w-full max-w-sm bg-white border border-neutral-200 rounded-2xl p-8 shadow-sm"> <h1 className="text-2xl font-extrabold mb-1.5">Create your account</h1> <p className="text-neutral-600 text-sm mb-6">Get moving with Uthao.</p> <div className="grid grid-cols-2 bg-neutral-100 rounded-md p-1 mb-5" role="tablist" aria-label="Account type" > <button type="button" role="tab" aria-selected={form.role === ROLES.RIDER} className={`py-2.5 rounded text-sm font-bold transition-colors ${form.role === ROLES.RIDER ? "bg-white text-black shadow-sm" : "bg-transparent text-neutral-600"}`} onClick={() => selectRole(ROLES.RIDER)} > Rider </button> <button type="button" role="tab" aria-selected={form.role === ROLES.DRIVER} className={`py-2.5 rounded text-sm font-bold transition-colors ${form.role === ROLES.DRIVER ? "bg-white text-black shadow-sm" : "bg-transparent text-neutral-600"}`} onClick={() => selectRole(ROLES.DRIVER)} > Driver </button> </div> <form onSubmit={handleSubmit}> <Input id="name" name="name" label="Full name" value={form.name} onChange={handleChange} required /> <Input id="email" name="email" label="Email" type="email" value={form.email} onChange={handleChange} required /> <Input id="password" name="password" label="Password" type="password" value={form.password} onChange={handleChange} required /> {error && ( <p className="bg-red-50 text-red-600 px-3 py-2.5 rounded-md text-sm -mt-1 mb-4"> {error} </p> )} <Button type="submit" disabled={loading} full> {loading ? "Creating account…" : "Create account"} </Button> </form> <p className="text-center text-sm text-neutral-600 mt-4"> Already have an account?{" "} <Link to="/login" className="text-black font-bold no-underline"> Log in </Link> </p> </div> </div> ); }
```

`src/pages/rider/RiderDashboard.jsx`

Shows `id`/`status`/`createdAt` only — `RideRequestResponse` doesn't return pickup/drop coordinates, so the list doesn't invent them.

```
import { useEffect, useState } from "react"; import { Link } from "react-router-dom"; import { useAuth } from "../../hooks/useAuth"; import { getRideRequests, cancelRideRequest } from "../../api/riderApi"; import Card from "../../components/Card"; import Button from "../../components/Button"; import StatusBadge from "../../components/StatusBadge"; import Loader from "../../components/Loader"; export default function RiderDashboard() { const { user } = useAuth(); const [requests, setRequests] = useState([]); const [loading, setLoading] = useState(true); async function load() { setLoading(true); const data = await getRideRequests(user.userId); setRequests([...data].sort((a, b) => new Date(b.createdAt) - new Date(a.createdAt))); setLoading(false); } useEffect(() => { load(); }, []); async function handleCancel(requestId) { await cancelRideRequest(requestId); load(); } return ( <div className="flex flex-col gap-4"> <div className="flex flex-col sm:flex-row sm:items-end sm:justify-between gap-4 mb-1"> <div> <p className="uppercase text-[11px] font-bold tracking-wider text-neutral-400 mb-1"> Rider </p> <h1 className="text-2xl font-extrabold">Your rides</h1> </div> <Link to="/rider/new"> <Button>Request a ride</Button> </Link> </div> <Card> {loading ? ( <Loader /> ) : requests.length === 0 ? ( <p className="text-neutral-400 text-sm text-center py-6"> No ride requests yet. Request your first ride. </p> ) : ( <ul className="list-none p-0 m-0"> {requests.map((r) => ( <li key={r.id} className="flex items-center justify-between gap-3 py-4 border-b border-neutral-200 last:border-b-0" > <div> <p className="text-[15px] font-semibold m-0">Ride request #{r.id}</p> <p className="text-[13px] text-neutral-400 mt-0.5"> {new Date(r.createdAt).toLocaleString()} </p> </div> <div className="flex items-center gap-3"> <StatusBadge status={r.status} /> {r.status === "REQUESTED" && ( <Button variant="secondary" onClick={() => handleCancel(r.id)}> Cancel </Button> )} </div> </li> ))} </ul> )} </Card> </div> ); }
```

`src/pages/rider/NewRideRequest.jsx`

Creates the ride request, asks matching-service to find a driver, then **polls trip-service's `GET /trips/by-request/{rideRequestId}`** every 1.5s until the async `driver.assigned` listener has created the trip row, then navigates straight to `TripDetails`. Pickup/drop are no longer raw numeric inputs — riders don't know their destination's latitude/longitude, so both fields are tap-to-open buttons that launch `LocationPickerModal`; pickup also keeps a "use current location" shortcut via the browser Geolocation API. The submit button is disabled until both points are set (`!pickup || !drop`), which replaces the old per-field `required` validation. Once pickup is set, `NearbyDriversMap` renders below the fields so the rider can see actual available drivers before submitting.

```
import { useState, useEffect, useRef } from "react"; import { useNavigate, Link } from "react-router-dom"; import { useAuth } from "../../hooks/useAuth"; import { createRideRequest } from "../../api/riderApi"; import { findDriver } from "../../api/matchingApi"; import { getTripByRideRequest } from "../../api/tripApi"; import Card from "../../components/Card"; import Button from "../../components/Button"; import LocationPickerModal from "../../components/LocationPickerModal"; import NearbyDriversMap from "../../components/NearbyDriversMap"; const POLL_INTERVAL_MS = 1500; function LocationField({ label, point, onPick, extra }) { return ( <div className="mb-4"> <div className="flex items-center justify-between mb-1.5"> <span className="text-xs font-semibold text-neutral-600">{label}</span> {extra} </div> <button type="button" onClick={onPick} className="w-full flex items-center justify-between px-3.5 py-3 border-[1.5px] border-neutral-200 rounded-md text-[15px] text-left hover:border-black transition-colors" > <span className={point ? "text-black" : "text-neutral-400"}> {point ? `${point.lat.toFixed(5)}, ${point.lng.toFixed(5)}` : "Tap to choose on map"} </span> <span className="text-neutral-400">▦</span> </button> </div> ); } export default function NewRideRequest() { const { user } = useAuth(); const navigate = useNavigate(); const [pickup, setPickup] = useState(null); const [drop, setDrop] = useState(null); const [pickerFor, setPickerFor] = useState(null); // "pickup" | "drop" | null const [match, setMatch] = useState(null); const [submitting, setSubmitting] = useState(false); const [waitingForTrip, setWaitingForTrip] = useState(false); const [error, setError] = useState(""); const pollRef = useRef(null); // Stop polling if the rider navigates away before the trip is ready. useEffect(() => { return () => clearInterval(pollRef.current); }, []); function useCurrentLocationForPickup() { navigator.geolocation.getCurrentPosition((position) => { setPickup({ lat: position.coords.latitude, lng: position.coords.longitude }); }); } function
```

```
pollForTrip(rideRequestId) { setWaitingForTrip(true); pollRef.current = setInterval(async () => { try { const trip = await getTripByRideRequest(rideRequestId); clearInterval(pollRef.current); setWaitingForTrip(false); navigate(`/trips/${trip.id}`); } catch { // 404 = trip-service hasn't consumed driver.assigned yet, keep polling. } }, POLL_INTERVAL_MS); } async function handleSubmit(e) { e.preventDefault(); setError(""); setSubmitting(true); try { const coords = { pickupLat: pickup.lat, pickupLng: pickup.lng, dropLat: drop.lat, dropLng: drop.lng, }; const request = await createRideRequest({ riderId: user.userId, ...coords }); const result = await findDriver({ rideRequestId: request.id, riderId: user.userId, ...coords, }); setMatch(result); if (result.status === "MATCHED") { pollForTrip(request.id); } } catch { setError("Could not create the ride request. Try again."); } finally { setSubmitting(false); } } return ( <div className="flex flex-col gap-4"> <div className="flex flex-col sm:flex-row sm:items-end sm:justify-between gap-4 mb-1"> <div> <p className="uppercase text-[11px] font-bold tracking-wider text-neutral-400 mb-1"> Rider </p> <h1 className="text-2xl font-extrabold">Request a ride</h1> </div> <Link to="/rider" className="inline-flex items-center justify-center rounded-full text-sm font-bold px-3 py-2 bg-transparent text-black hover:bg-neutral-100 transition-colors no-underline" > Back </Link> </div> <Card> <form onSubmit={handleSubmit}> <LocationField label="Pickup" point={pickup} onPick={() => setPickerFor("pickup")} extra={ <button type="button" className="text-blue-600 text-xs font-semibold cursor-pointer" onClick={useCurrentLocationForPickup} > Use current location </button> } /> <LocationField label="Drop-off" point={drop} onPick={() => setPickerFor("drop")} /> <NearbyDriversMap pickup={pickup} /> {error && ( <p className="bg-red-50 text-red-600 px-3 py-2.5 rounded-md text-sm mb-4">{error}</p> )} <Button type="submit" disabled={submitting || !pickup || !drop} full> {submitting ? "Finding a driver…" : "Request ride"} </Button> </form> </Card> {match && ( <Card title="Match result"> {match.status === "MATCHED" ? ( <div className="flex items-center gap-3.5"> <span className="inline-flex items-center justify-center w-11 h-11 rounded-full bg-black text-white text-lg font-bold shrink-0"> {match.driverName?.charAt(0).toUpperCase()} </span> <div> <p className="text-[15px] font-semibold m-0">{match.driverName}</p> <p className="text-[13px] text-neutral-400 mt-0.5"> {match.vehiclePlate} · ETA {match.eta} </p> {waitingForTrip && ( <p className="text-[13px] text-neutral-400 mt-0.5">Opening trip…</p> )} </div> </div> ) : ( <p className="text-neutral-400 text-sm text-center py-6"> No driver available right now. Try again shortly. </p> )} </Card> )} {pickerFor && ( <LocationPickerModal title={pickerFor === "pickup" ? "Choose pickup location" : "Choose drop-off location"} initial={pickerFor === "pickup" ? pickup : drop} onConfirm={(point) => { if (pickerFor === "pickup") { setPickup(point); } else { setDrop(point); } setPickerFor(null); }} onClose={() => setPickerFor(null)} /> )} </div> );
```

`src/pages/driver/DriverOnboarding.jsx`

Guards against re-registering: if `driverId` is already known (context/localStorage), it redirects to `/driver` immediately instead of letting the form submit again — see §16 for why that matters (`POST /drivers` has no uniqueness check). Prefills the email field from `GET /auth/me` since `AuthContext.user` doesn't carry email, only `userId`/`name`/`role`.

```
import { useEffect, useState } from "react"; import { useNavigate } from "react-router-dom"; import { useAuth } from "../../hooks/useAuth"; import { getCurrentUser } from "../../api/authApi"; import { registerDriver, addVehicle } from "../../api/driverApi"; import Card from "../../components/Card"; import Input from "../../components/Input"; import Button from "../../components/Button"; export default function DriverOnboarding() { const { user, driverId, completeDriverOnboarding } = useAuth(); const navigate = useNavigate(); const [profile, setProfile] = useState({ email: "", phone: "", licenseNumber: "" }); const [vehicle, setVehicle] = useState({ make: "", model: "", plateNumber: "", color: "" }); const [submitting, setSubmitting] = useState(false); const [error, setError] = useState(""); // Already onboarded (e.g. revisited this URL directly) — don't let them // submit a second Driver row for the same userId. useEffect(() => { if (driverId) { navigate("/driver", { replace: true }); } }, [driverId]); useEffect(() => { getCurrentUser() .then((me) => setProfile((p) => ({ ...p, email: me.email }))) .catch(() => {}); }, []); async function handleSubmit(e) { e.preventDefault(); setError(""); setSubmitting(true); try { const driver = await registerDriver({ userId: user.userId, name: user.name, email: profile.email, phone: profile.phone, licenseNumber: profile.licenseNumber, }); await addVehicle(driver.id,
```

```
vehicle); completeDriverOnboarding(driver.id); navigate("/driver"); } catch { setError("Could not complete onboarding. Check your details and try again."); } finally { setSubmitting(false); } } return ( <div className="flex flex-col gap-4"> <div className="flex flex-col sm:flex-row sm:items-end sm:justify-between gap-4 mb-1"> <div> <p className="uppercase text-[11px] font-bold tracking-wider text-neutral-400 mb-1"> Driver </p> <h1 className="text-2xl font-extrabold">Finish setting up</h1> </div> </div> <Card title="Your details"> <form onSubmit={handleSubmit}> <Input label="Email" type="email" value={profile.email} onChange={(e) => setProfile({ ...profile, email: e.target.value })} required /> <Input label="Phone" value={profile.phone} onChange={(e) => setProfile({ ...profile, phone: e.target.value })} required /> <Input label="License number" value={profile.licenseNumber} onChange={(e) => setProfile({ ...profile, licenseNumber: e.target.value })} required /> <p className="uppercase text-[11px] font-bold tracking-wider text-neutral-400 mb-1"> Vehicle </p> <div className="grid grid-cols-1 sm:grid-cols-2 gap-3"> <Input label="Make" value={vehicle.make} onChange={(e) => setVehicle({ ...vehicle, make: e.target.value })} required /> <Input label="Model" value={vehicle.model} onChange={(e) => setVehicle({ ...vehicle, model: e.target.value })} required /> </div> <div className="grid grid-cols-1 sm:grid-cols-2 gap-3"> <Input label="Plate number" value={vehicle.plateNumber} onChange={(e) => setVehicle({ ...vehicle, plateNumber: e.target.value })} required /> <Input label="Color" value={vehicle.color} onChange={(e) => setVehicle({ ...vehicle, color: e.target.value })} required /> </div> {error && ( <p className="bg-red-50 text-red-600 px-3 py-2.5 rounded-md text-sm -mt-1 mb-4"> {error} </p> )} <Button type="submit" disabled={submitting} full> {submitting ? "Saving…" : "Finish onboarding"} </Button> </form> </Card> </div> ); }
```

`src/pages/driver/DriverDashboard.jsx`

Uses `driverId` from context (never `user.userId`) for every driver-service call. `getDriverTrips` returns driver-service's own `DriverTrip` records — these only ever hold `id`, `driverId`, `rideRequestId`, `riderId`, `eta`, `assignedAt` (**no `tripId`, no status, no fare** — that data lives in trip-service's `Trip` entity instead), so each row resolves the real trip on demand via `getTripByRideRequest` before navigating. `status` is fetched from the backend on mount (`getDriverStatus`) rather than assumed — it used to hardcode to `OFFLINE` on every mount, so a page refresh always showed offline even if the driver was actually `AVAILABLE` server-side. A second effect re-pushes fresh coordinates on an interval for as long as `status` stays `AVAILABLE` — toggling status only sends one lat/lng snapshot at that instant, which would go stale the moment the driver moves, and `NearbyDriversMap.jsx` on the rider side refreshes every 10s expecting that live data.

```
import { useEffect, useState } from "react"; import { useNavigate } from "react-router-dom"; import { useAuth } from "../../hooks/useAuth"; import { updateDriverStatus, getDriverStatus, getDriverTrips } from "../../api/driverApi"; import { getTripByRideRequest } from "../../api/tripApi"; import Card from "../../components/Card"; import Button from "../../components/Button"; import StatusBadge from "../../components/StatusBadge"; import Loader from "../../components/Loader"; import { DRIVER_STATUS } from "../../utils/constants"; const LOCATION_PUSH_INTERVAL_MS = 20000; export default function DriverDashboard() { const { driverId } = useAuth(); const navigate = useNavigate(); const [status, setStatus] = useState(DRIVER_STATUS.OFFLINE); const [trips, setTrips] = useState([]); const [loading, setLoading] = useState(true); const [updatingStatus, setUpdatingStatus] = useState(false); useEffect(() => { if (!driverId) { navigate("/driver/onboarding", { replace: true }); return; } loadStatus(); loadTrips(); }, [driverId]); // Toggling AVAILABLE only sends one lat/lng snapshot at that instant — if the // driver keeps moving, riders searching nearby would see a stale position. // Keep pushing fresh coordinates on an interval for as long as status stays // AVAILABLE; stop the moment it flips to OFFLINE or the page unmounts. useEffect(() => { if (!driverId || status !== DRIVER_STATUS.AVAILABLE) { return; } const interval = setInterval(() => { navigator.geolocation.getCurrentPosition((position) => { updateDriverStatus(driverId, { status: DRIVER_STATUS.AVAILABLE, currentLat: position.coords.latitude, currentLng: position.coords.longitude, }); }); }, LOCATION_PUSH_INTERVAL_MS); return () => clearInterval(interval); }, [driverId, status]); async function loadStatus() { const data = await getDriverStatus(driverId); setStatus(data.status); } async function loadTrips() { setLoading(true); const data = await getDriverTrips(driverId); setTrips(data); setLoading(false); } function toggleStatus() { const next = status ===
```

```
DRIVER_STATUS.AVAILABLE ? DRIVER_STATUS.OFFLINE : DRIVER_STATUS.AVAILABLE;
setUpdatingStatus(true); navigator.geolocation.getCurrentPosition( async (position) => {
await updateDriverStatus(driverId, { status: next, currentLat: position.coords.latitude,
currentLng: position.coords.longitude, }); setStatus(next); setUpdatingStatus(false); }, ()
=> setUpdatingStatus(false) ); } async function openTrip(rideRequestId) { try { const trip =
await getTripByRideRequest(rideRequestId); navigate( /trips/${trip.id} ); } catch {
alert("Trip isn't ready yet — try again in a moment."); } } if (!driverId) { return null; }
return ( <div className="flex flex-col gap-4"> <div className="flex flex-col sm:flex-row
sm:items-end sm:justify-between gap-4 mb-1"> <div> <p className="uppercase text-[11px]
font-bold tracking-wider text-neutral-400 mb-1"> Driver </p> <h1 className="text-2xl
font-extrabold">Dashboard</h1> </div> </div> <Card> <div className="flex items-center
justify-between gap-4"> <div> <p className="text-[15px] font-semibold m-0">Your status</p>
<StatusBadge status={status} /> {status === DRIVER_STATUS.AVAILABLE && ( <p
className="text-[12px] text-neutral-400 mt-1">Sharing your location live</p> )} </div>
<Button onClick={toggleStatus} disabled={updatingStatus} variant={status ===
DRIVER_STATUS.AVAILABLE ? "secondary" : "primary"} > {updatingStatus ? "Updating…" : status
=== DRIVER_STATUS.AVAILABLE ? "Go offline" : "Go available"} </Button> </div> </Card> <Card
title="Trip history"> {loading ? ( <Loader /> ) : trips.length === 0 ? ( <p
className="text-neutral-400 text-sm text-center py-6"> No trips yet. Go available to start
receiving rides. </p> ) : ( <ul className="list-none p-0 m-0"> {trips.map((t) => ( <li
key={t.id} className="flex items-center justify-between gap-3 py-4 border-b
border-neutral-200 last:border-b-0" > <div> <p className="text-[15px] font-semibold m-0">
Ride request #{t.rideRequestId} </p> <p className="text-[13px] text-neutral-400 mt-0.5"> ETA
{t.eta} · {new Date(t.assignedAt).toLocaleString()} </p> </div> <Button variant="secondary"
onClick={() => openTrip(t.rideRequestId)}> Open trip </Button> </li> ))} </ul> )} </Card>
</div> ); }
```

`src/pages/trip/TripDetails.jsx`

Fetches payment only once the trip is `COMPLETED` (that's the only trip state payment-service has a row for), and refunds using `payment.id` — not `trip.id` — with a required `reason` in the body.

`Start trip`/`Complete trip` only render for `user.role === "DRIVER"`; `Cancel trip` only renders while `trip.status === "MATCHED"` (before the trip has started), regardless of role. This mirrors real enforcement added to trip-service itself (§16) — hiding the buttons is UX, not the actual security boundary, so `handleAction` also catches a 403 (wrong role) or 409 (trip already started) from the backend and shows a message, covering the case where the UI's local state is stale (e.g. two tabs open).

```
import { useEffect, useState } from "react"; import { useParams, Link } from
"react-router-dom"; import { useAuth } from "../../hooks/useAuth"; import { getTrip,
startTrip, completeTrip, cancelTrip } from "../../api/tripApi"; import { getPaymentByTrip,
refundPayment } from "../../api/paymentApi"; import Card from "../../components/Card";
import Button from "../../components/Button"; import StatusBadge from
"../../components/StatusBadge"; import Loader from "../../components/Loader"; export default
function TripDetails() { const { tripId } = useParams(); const { user } = useAuth(); const
[trip, setTrip] = useState(null); const [payment, setPayment] = useState(null); const
[loading, setLoading] = useState(true); const [actionPending, setActionPending] =
useState(false); const [actionError, setActionError] = useState(""); async function load() {
const data = await getTrip(tripId); setTrip(data); if (data.status === "COMPLETED") { const
paymentData = await getPaymentByTrip(tripId).catch(() => null); setPayment(paymentData); }
else { setPayment(null); } setLoading(false); } useEffect(() => { setLoading(true); load();
}, [tripId]); async function handleAction(action) { setActionError(""); }
setActionPending(true); try { await action(tripId); await load(); } catch (err) {
setActionError( err.response?.status === 403 ? "Only the driver can do that." : "That action
isn't allowed right now." ); } finally { setActionPending(false); } } async function
handleRefund() { if (!payment) return; setActionPending(true); try { await
refundPayment(payment.id, "Rider requested refund"); await load(); } finally {
setActionPending(false); } } if (loading || !trip) return <Loader />; return ( <div
className="flex flex-col gap-4"> <div className="flex flex-col sm:flex-row sm:items-end
sm:justify-between gap-4 mb-1"> <div> <p className="uppercase text-[11px] font-bold
tracking-wider text-neutral-400 mb-1"> Trip #{trip.id} </p> <h1> <StatusBadge
```

```
status={trip.status} /> </h1> </div> <Link to="/" className="inline-flex items-center justify-center rounded-full text-sm font-bold px-3 py-2 bg-transparent text-black hover:bg-neutral-100 transition-colors no-underline" > Home </Link> </div> <Card title="Details"> <p className="text-[13px] text-neutral-400"> Driver #{trip.driverId} · Rider #{trip.riderId} </p> {trip.status === "COMPLETED" && trip.fare != null && ( <p className="text-2xl font-extrabold mt-2">₹{trip.fare.toFixed(2)}</p> )} <div className="flex gap-3 mt-5"> {trip.status === "MATCHED" && user.role === "DRIVER" && ( <Button onClick={() => handleAction(startTrip)} disabled={actionPending}> Start trip </Button> )} {trip.status === "ONGOING" && user.role === "DRIVER" && ( <Button onClick={() => handleAction(completeTrip)} disabled={actionPending}> Complete trip </Button> )} {trip.status === "MATCHED" && ( <Button variant="secondary" onClick={() => handleAction(cancelTrip)} disabled={actionPending} > Cancel trip </Button> )} </div> {actionError && ( <p className="bg-red-50 text-red-600 px-3 py-2.5 rounded-md text-sm mt-3"> {actionError} </p> )} </Card> {payment && ( <Card title="Payment"> <div className="flex items-center justify-between gap-4"> <p className="text-2xl font-extrabold">₹{payment.amount.toFixed(2)}</p> <StatusBadge status={payment.status} /> </div> {payment.status === "SUCCESS" && ( <Button variant="secondary" onClick={handleRefund} disabled={actionPending} className="mt-4" > Request refund </Button> )} </Card> )} </div> ); }
```

src/pages/NotificationsPage.jsx

```
import { useEffect } from "react"; import { useAuth } from "../hooks/useAuth"; import { useNotifications } from "../hooks/useNotifications"; import Card from "../components/Card"; import Button from "../components/Button"; import Loader from "../components/Loader"; export default function NotificationsPage() { const { user } = useAuth(); const { notifications, fetchNotifications, markRead, loading } = useNotifications(); useEffect(() => { fetchNotifications(user.userId); }, [user.userId]); return ( <div className="flex flex-col gap-4"> <div className="flex flex-col sm:flex-row sm:items-end sm:justify-between gap-4 mb-1"> <div> <p className="uppercase text-[11px] font-bold tracking-wider text-neutral-400 mb-1"> Inbox </p> <h1 className="text-2xl font-extrabold">Notifications</h1> </div> </div> <Card> {loading ? ( <Loader /> ) : notifications.length === 0 ? ( <p className="text-neutral-400 text-sm text-center py-6">No notifications yet.</p> ) : ( <ul className="list-none p-0 m-0"> {notifications.map((n) => ( <li key={n.id} className="relative flex items-center justify-between gap-3 py-4 border-b border-neutral-200 last:border-b-0" > {!n.isRead && ( <span className="absolute -left-3.5 top-6 w-1.5 h-1.5 rounded-full bg-blue-600" /> )} <div> <p className="text-[15px] font-semibold m-0">{n.message}</p> <p className="text-[13px] text-neutral-400 mt-0.5"> {new Date(n.createdAt).toLocaleString()} </p> </div> {!n.isRead && ( <Button variant="secondary" onClick={() => markRead(n.id)}> Mark read </Button> )} </li> ))} </ul> )} </Card> </div> ); }
```

src/pages/NotFoundPage.jsx

```
import { Link } from "react-router-dom"; export default function NotFoundPage() { return ( <div className="text-center py-24 px-4"> <h1 className="text-6xl font-extrabold">404</h1> <p className="text-neutral-600 my-2 mb-6">We couldn't find that page.</p> <Link to="/" className="inline-flex items-center justify-center rounded-full text-sm font-bold px-6 py-3.5 bg-black text-white hover:bg-neutral-800 transition-colors no-underline" > Go home </Link> </div> ); }
```

---

13. Styling — Tailwind CSS v4

Tailwind v4 dropped the `tailwind.config.js` / `postcss.config.js` requirement for a default theme — the whole setup is one Vite plugin plus one `@import`.

`vite.config.js`

```
import { defineConfig } from "vite"; import react from "@vitejs/plugin-react"; import tailwindcss from "@tailwindcss/vite"; export default defineConfig({ plugins: [react(), tailwindcss()], server: { port: 5173, }, });
```

`src/index.css`

```
@import "tailwindcss"; body { @apply bg-neutral-100 text-black antialiased; }
```

That's the entire stylesheet. Everything else is utility classes directly in JSX `className` props (see §11–12 for every component/page). Two patterns worth calling out:

- **Dynamic classes use a literal lookup object, never string interpolation into a utility name.** Tailwind's compiler scans source files for complete class-name strings at build time; `` `bg-${status}-50` `` never resolves to real CSS because the scanner can't see the concatenated result. `StatusBadge.jsx` and `Button.jsx` both use a `{ KEY: "full literal classes" }` map instead — every branch is a real, scannable string.
- **No custom theme extension.** Tailwind's default palette (`neutral`, `green`, `amber`, `red`, `blue`), spacing scale, and font stack were close enough to the Uber-style target that no `@theme` customization was needed — arbitrary-value syntax (`text-[15px]`, `min-h-[calc(100vh-65px)]`) covers the few one-off values.

---

14. Backend Endpoint Reference

All requests go through the gateway at `http://localhost:8080/api`. Every route except `/api/auth/**` requires `Authorization: Bearer <token>`.

Frontend API function	Method & Path	Service
`registerUser`	`POST /api/auth/register`	identity-service
`loginUser`	`POST /api/auth/login`	identity-service
`getCurrentUser`	`GET /api/auth/me`	identity-service
`createRideRequest`	`POST /api/riders/requests`	rider-service
`getRideRequests`	`GET /api/riders/{riderId}/requests`	rider-service
`cancelRideRequest`	`POST /api/riders/requests/{requestId}/cancel`	rider-service
`registerDriver`	`POST /api/drivers`	driver-service

`addVehicle`	`POST /api/drivers/{driverId}/vehicles`	driver-service
`updateDriverStatus`	`PATCH /api/drivers/{driverId}/status`	driver-service
`getDriverStatus`	`GET /api/drivers/{driverId}/status`	driver-service
`getAvailableDrivers`	`GET /api/drivers/available?lat=&lng=&radiusKm=`	driver-service
`getDriver`	`GET /api/drivers/{driverId}`	driver-service
`getDriverByUserId`	`GET /api/drivers/by-user/{userId}`	driver-service
`getDriverTrips`	`GET /api/drivers/{driverId}/trips`	driver-service
`findDriver`	`POST /api/matching/find-driver`	matching-service
`getTripByRideRequest`	`GET /api/trips/by-request/{rideRequestId}`	trip-service
`getTrip`	`GET /api/trips/{tripId}`	trip-service
`startTrip`	`POST /api/trips/{tripId}/start`	trip-service
`completeTrip`	`POST /api/trips/{tripId}/complete`	trip-service
`cancelTrip`	`POST /api/trips/{tripId}/cancel`	trip-service
`getPaymentByTrip`	`GET /api/payments/trip/{tripId}`	payment-service
`refundPayment`	`POST /api/payments/{paymentId}/refund`	payment-service
`getNotifications`	`GET /api/notifications/user/{userId}`	notification-service

`markNotificationRead`	`PATCH /api/notifications/{id}/read`	notification-service

Full JSON shapes are in `Docs/CONTRACTS.md` §6. The two rows in *italics-worthy* bold above (`getDriverByUserId`, `getTripByRideRequest`) call endpoints that were added to the backend specifically to support this frontend — see §16.

---

15. Running the App

1. Start the backend — Postgres + RabbitMQ (`docker compose up -d`), then `eureka-server`, then every `*-service`, then `api-gateway` last (see the root `README.md` for the exact order). The gateway must be reachable at `http://localhost:8080`. `api-gateway` needs `config/CorsConfig.java` (a `CorsWebFilter` allowing any `localhost:*` origin) — without it the browser blocks every request at the CORS preflight stage even though curl/Postman work fine, since those don't send preflight `OPTIONS` requests. Already in the repo; just make sure you're not running an older build of `api-gateway`.

   Install dependencies once:

```
cd uthao-frontend npm install
```

   Start the dev server:

```
npm run dev
```

   Open `http://localhost:5173`.

5. Register as a `RIDER` and a `DRIVER` (two different browser sessions/incognito windows work well), finish driver onboarding, set the driver `AVAILABLE`, then create a ride request as the rider and click "Request ride" to trigger matching.

To verify the build without running the dev server (e.g. before a deploy):

```
npm run build # production bundle to dist/ — also catches import/JSX errors npm run preview
# serve that production build locally
```

---

16. Known Limitations & Next Steps

These mirror gaps already called out in the backend docs (`README.md` lists "WebSocket, live tracking, rating service, frontend" as out of scope for this phase), plus backend gaps this frontend project surfaced and fixed directly:

Trip start/complete/cancel had zero authorization — any valid JWT could call any of them. Before this fix, **no downstream service parsed the JWT at all**; the gateway only checked the token was validly signed, never who it belonged to. A rider could `POST /trips/{tripId}/start` directly (via curl/Postman, bypassing the UI) exactly as freely as the actual driver. Fixed in trip-service:

- Added `com.uthao.trip.security.JwtUtil` (mirrors identity-service's own `JwtUtil`) plus the `jjwt` dependency and `jwt.secret` config, neither of which trip-service had before.

- `TripController` now requires the `Authorization` header on `/start` and `/complete` and passes the decoded `role` claim to `TripService`, which throws `403` via `requireDriver()` unless `role == "DRIVER"`.
- `TripService.cancelTrip` now throws `409 Conflict` unless the trip is still `MATCHED` — cancellation is only valid *before* a trip starts, for either role.
- Verified live (not just "compiles"): a rider token gets `403` on start/complete and `409` cancelling an already-`ONGOING` trip; a driver token passes through to the normal 404-if-trip-doesn't-exist path. The same three assertions are now automated in the Postman collection's `05 - Trip Lifecycle` folder.
- `TripDetails.jsx` hides the Start/Complete buttons unless `user.role === "DRIVER"` and hides Cancel once `status !== "MATCHED"` — but that's UX, not the security boundary; the backend enforces it regardless of what the UI shows, and `handleAction` surfaces a message if a stale UI still lets someone try.

- **Trip lookup, solved by polling, not push.** `driver.assigned` → trip creation happens asynchronously over RabbitMQ, so the moment `find-driver` returns `MATCHED` the trip row may not exist yet, and its response never carries a `tripId`. Fixed by adding **`GET /trips/by-request/{rideRequestId}`** to trip-service (`TripRepository.findByRideRequestId`, `TripService.getTripByRideRequestId`, `TripController`), which 404s until the listener has created the row. `NewRideRequest.jsx` polls it every 1.5s after a match; `DriverDashboard.jsx` calls it on demand when opening a trip from history. This is a short polling loop, not a live channel — swap in a WebSocket/SSE push if trip creation ever gets slow or you want true real-time tracking (e.g. live driver location during the ride).
- **"See nearby drivers" is pre-request only, not en-route tracking.** `NearbyDriversMap.jsx` (§11) shows *available, unassigned* drivers near the pickup point before the rider submits a request — it stops being relevant the moment a match happens, since matched/on-trip drivers no longer show up in `GET /drivers/available` (it only ever returns `status = AVAILABLE`). Watching *your assigned driver's* live position during an active trip is a different, unbuilt feature — it would need a location field on `Trip`/`DriverTrip` updated continuously and polled or pushed to `TripDetails.jsx`, not just the pickup-time driver search this reuses.
- **Driver identity resolution.** Every driver-service endpoint (`status`, `vehicles`, `trips`) is keyed by driver-service's own `Driver.id`, not the identity-service `userId` — but the only way to learn that id was the response of `POST /drivers` at registration time. A driver logging in again in a new session/browser had no way to recover it. Fixed by adding **`GET /drivers/by-user/{userId}`** (`DriverRepository.findByUserId`, `DriverService.getDriverByUserId`, `DriverController`). `AuthContext.login()` now resolves and caches `driverId` automatically; `register()` doesn't bother (a brand-new account can't have one yet).
- **Driver status was write-only.** The backend originally had `PATCH /drivers/{driverId}/status` to set status but no way to read it back, so `DriverDashboard.jsx` hardcoded `status` to `OFFLINE` in local state on every mount — a page refresh always showed offline regardless of what was actually stored server-side. Fixed by adding **`GET /drivers/{driverId}/status`** (`DriverService.getStatus`, `DriverController`), which defaults to `OFFLINE` rather than 404ing when no `DriverStatus` row exists yet (true for every driver who hasn't toggled status once). `DriverDashboard.jsx` now fetches it on mount.
- **`POST /drivers` has no uniqueness constraint on `userId`.** Nothing at the database level stops two `Driver` rows being created for the same user — if that ever happened, `GET /drivers/by-user/{userId}` (a `findByUserId` returning a single `Optional`) would throw on the duplicate. `DriverOnboarding.jsx` guards the common path (redirects away if `driverId` is already known) but this isn't a DB-level fix; a real fix would add a unique index on `drivers.user_id`.
- **No token expiry/refresh.** The backend JWT has no expiry in this phase, so the frontend doesn't need refresh-token logic — just the 401 interceptor already shown.
- **No form validation library.** Inputs use plain HTML `required` — fine for a minimal internal tool, swap in a validation lib only if the project grows.

- **Location picking is pin-drop only, no address search.** `LocationPickerModal.jsx` solves "the rider doesn't know their destination's lat/lng" but not "the rider wants to type an address" — there's no geocoding (e.g. Nominatim search-by-name) wired in. Adding it would mean a search box in the modal that calls `https://nominatim.openstreetmap.org/search` and re-centers the map on the result — free, no API key, same OpenStreetMap data already in use for tiles.
- **Optional next step:** add a small polling `useEffect` (every few seconds) on `TripDetails` itself to refresh status live while a trip is `ONGOING`, instead of only refreshing after a button click.